

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP HCM
KHOA TOÁN - TIN HỌC
BỘ MÔN ỨNG DỤNG TIN HỌC

GIỚI THIỆU CHUNG

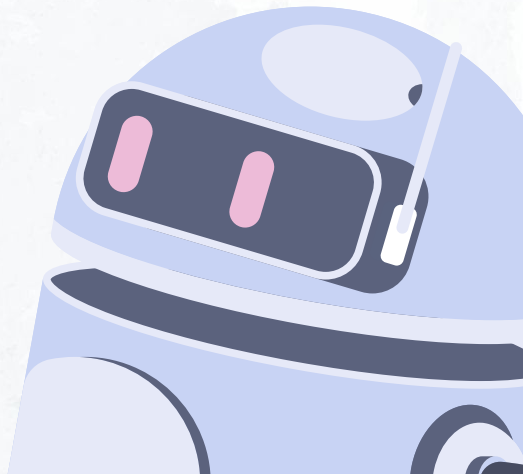


THỰC HÀNH CẤU TRÚC DỮ LIỆU
VÀ GIẢI THUẬT

TUẦN 01 - THÁNG 03 NĂM 2026



DSA



THÔNG TIN LIÊN HỆ

SAKAI: mcslearn.hcmus.edu.vn

EMAIL: ta.tintth.hcmus@gmail.com

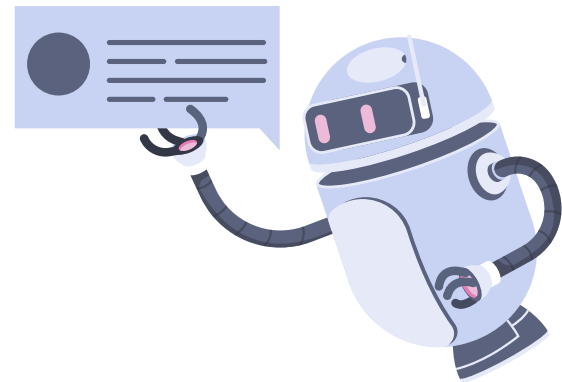
GHI CHÚ:

Tiêu đề mail (bắt buộc):

[CTDL26] [Tiêu đề thư]

VD: [CTDL25] HỎI BÀI

Gửi email kèm giới thiệu họ tên, MSSV và tên ca học.



NỘI DUNG

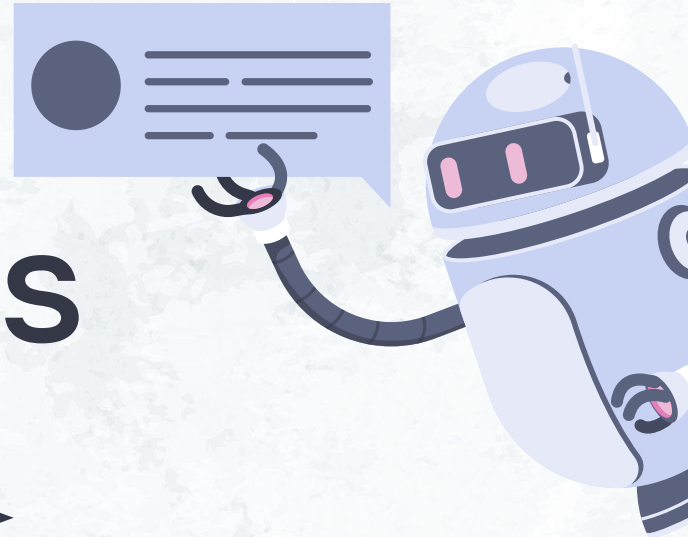
- 01 → GIỚI THIỆU MÔN HỌC
- 02 → NHẮC LẠI MỘT SỐ KIẾN THỨC VỀ MẢNG
- 03 → BÀI TẬP TẠI LỚP
- 04 → BÀI TẬP VỀ NHÀ

01 →

GIỚI THIỆU MÔN HỌC

(AI)

(DSA) =
Data Structures
& Algorithms →



QUY ĐỊNH BÀI TẬP THỰC HÀNH

Các bài tập thực hành được giao về nhà cần đáp ứng một số quy định sau:

1. **Bài nộp:** Chỉ nộp đúng một file ở dạng [Họ tên][MSSV].pdf
VD: PhamHungLam22110033.pdf
2. Trong file gồm **source code** + ảnh chụp màn hình **kết quả các Test Case** của đề và của cá nhân tự tạo.
3. Sử dụng ngôn ngữ **C** để xây dựng các source code. Source code xây dựng từ các ngôn ngữ khác C đều không được chấp nhận.
4. Các source code cần tuân theo cấu trúc sau:
 - a. Thư viện/hàng
 - b. Các hàm con xử lý
 - c. Hàm main
5. Mỗi hàm, mỗi bước làm phải có **chú thích cụ thể**. Source code không có chú thích thì được xem là chưa hợp lệ.
6. **Nơi nộp:** mục Assignment trên hệ thống sakai.

TÀI LIỆU THAM KHẢO

1. **A Common-Sense Guide to Data Structures and Algorithms** - Jay Wengrow
(bản pdf của sách có trên hệ thống sakai)
2. **Cấu trúc dữ liệu và giải thuật** - Phạm Thế Bảo
(sách có bán tại quầy giáo trình)
3. **In03 - Con trỏ và In04 - Mảng & Chuỗi và In05 - Hàm**
(Tài liệu Cơ sở lập trình - học kỳ 2 năm 2023)

Một số nội dung được học trong môn

(1) Tổng quan về Cấu trúc dữ liệu và giải thuật (mảng)

- Reading (Đọc)
- Searching (Tìm)
- Insertion (Chèn)
- Deletion (Xóa)

(4) Các thuật toán tìm kiếm và sắp xếp mảng

Search

- Binary search
- Linear search

Sort

- Bubble sort
- Selection sort
- Insertion sort

(5) Stack và Queue

- Khái niệm
- Bộ hàm cơ bản

(6) Đệ quy

- Khái niệm đệ quy
- Phân tích 4 bước
- Quick sort

(7) Cấu trúc Node - Base

Linked List

- Reading
- Searching
- Insertion
- Deletion

Binary tree

- Reading
- Searching
- Insertion
- Deletion

(3) Tổng quan về Big O

- Đếm số bước của thuật toán (comparison, assignment, shift,...)
- Thống kê.

Một số khái niệm

(a) Data (dữ liệu)

- Là một thuật ngữ đề cập đến các loại thông tin (information), kể cả số (number) và chuỗi/ chữ cơ bản (string)

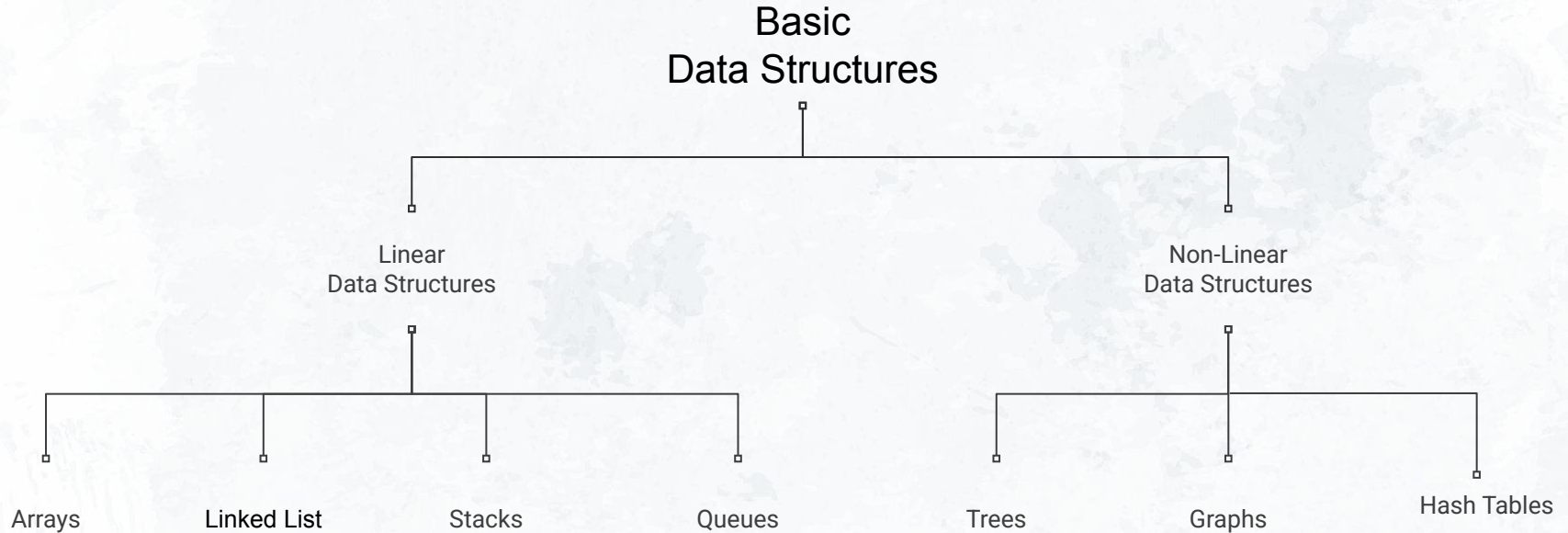
(b) Data structure (cấu trúc dữ liệu)

- Đề cập đến cách dữ liệu được tổ chức, một trong những yếu tố hỗ trợ cho chương trình. Thuật toán tối ưu
- 4 loại phổ biến được biết đến: Stack, Queue, Tree, Linked List

(c) Algorithm (thuật toán)

- Một chuỗi các hành động giải quyết một tập giá trị đầu vào để cho ra tập giá trị đầu ra tương ứng.
- 2 loại thuật toán chủ yếu sử dụng trong môn này: sorting, searching

Một số khái niệm



Array: cố định kích cỡ

Linked-list: kích cỡ có thể thay đổi được (biến kích cỡ)

Stack: Thêm vào ở đầu (top) và lấy ra từ đầu.

Queue: Thêm vào ở cuối (back) và lấy ra từ đầu.

Priority queue: Thêm vào ở bất cứ đâu, xóa ở nơi có mức độ ưu tiên cao nhất.

Hash tables: Các danh sách chưa được sắp thứ tự, sử dụng 'hash function' để chèn và tìm kiếm

Tree: Dữ liệu được tổ chức ở dạng phân nhánh.

Graph: Cấu trúc phân nhánh tổng quát hơn, với điều kiện kết nối ít chặt chẽ hơn so với **tree**.

Một số khái niệm

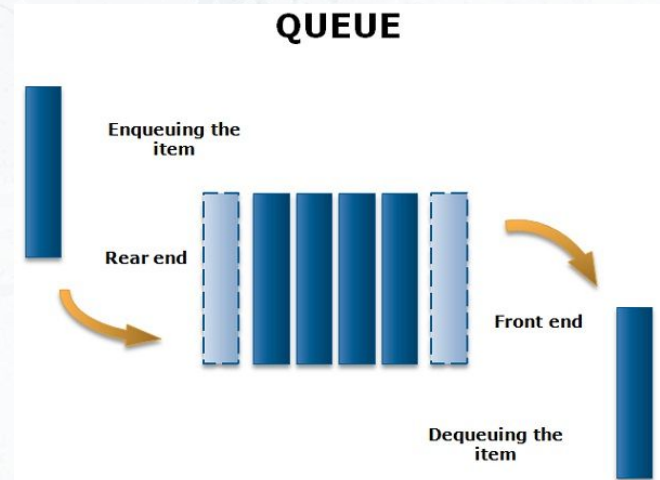
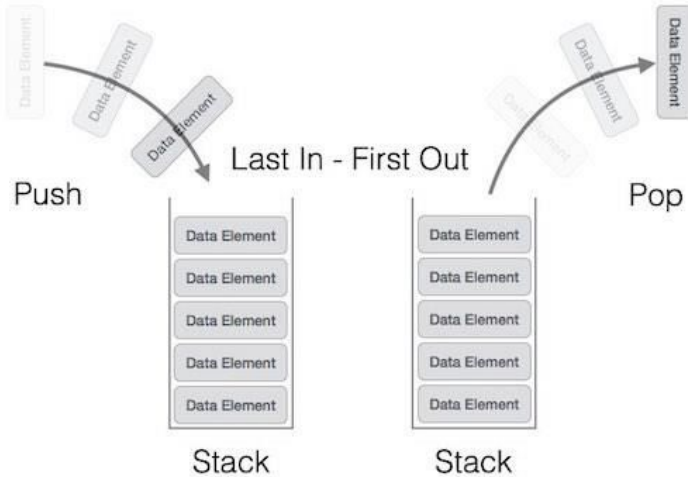
LINEAR DATA STRUCTURE



Array

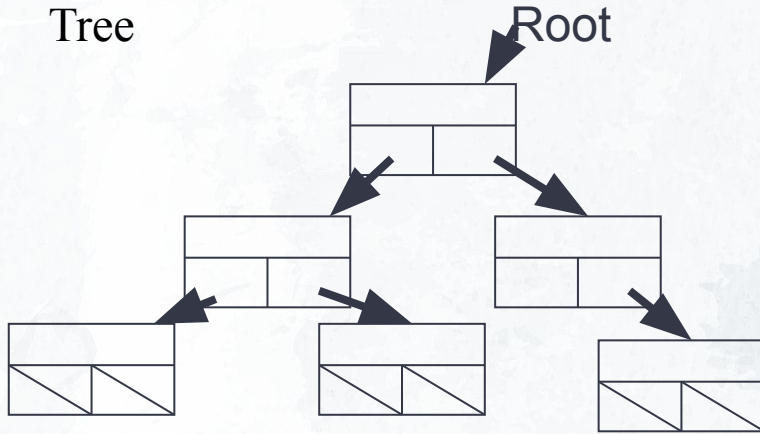


Linked list



Một số khái niệm

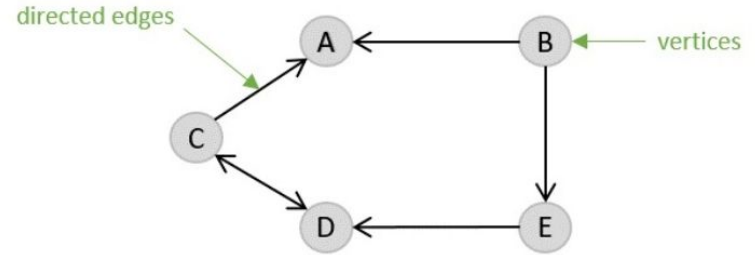
Tree



Hash table

keys	hash function	hash code
"YYZ"		5
"DTW"		0
"SFO"		4
"LHR"		1

NON - LINEAR DATA STRUCTURE



Graph Data Structure

0	"Detroit"
1	"London Heathrow"
2	
3	
4	"San Francisco"
5	"Toronto Pearson"
6	
7	

02 →

NHẮC LẠI MỘT SỐ KIẾN THỨC VỀ MẢNG

ARRAY

Nhắc lại một số kiến thức về mảng

(a) Mảng (arrays):

- Là một trong những loại data structure cơ bản
- Là một nhóm các vị trí bộ nhớ liên tiếp có cùng tên (name) và kiểu dữ liệu (data types)

(b) Phần tử của mảng (elements of arrays): Các vị trí bộ nhớ trong mảng được gọi là phần tử của mảng

(c) Độ dài của mảng (lengths of arrays): là số phần tử trong mảng

(d) Chỉ số *i* của mảng (index/ subscript of arrays): các phần tử của mảng được truy cập bằng cách tham chiếu đến vị trí của nó trong mảng gọi là các chỉ mục

Mảng khác gì với biến?

*“Biến là một vị trí bộ nhớ duy nhất, có tên và kiểu dữ liệu
Mảng là tập hợp nhiều vị trí bộ nhớ liên kế khác nhau”*

Nhắc lại một số kiến thức về mảng

(e) Công dụng của mảng:

- Lưu trữ lượng lớn giá trị với 1 tên
- Có thể xử lý, lưu trữ, sắp xếp, tìm kiếm nhiều giá trị dễ dàng và nhanh chóng

(f) Các loại mảng:

- Mảng 1 chiều (One-dimensional array)
- Mảng 2 chiều (Two-dimensional array)
- Mảng n chiều (Multi-dimensional array)

Mảng 1 chiều (One-dimensional array)

(a) Khai báo mảng 1D

- Khai báo 1-D Array: **data_type identifier[length];**
- Giải thích cú pháp:
 - data_type : kiểu dữ liệu của giá trị được lưu trữ trong mảng
 - Identifier: tên của mảng
 - length : số lượng phần tử
- Ví dụ

```
int marks[5]; \\khai báo mảng tên marks có 5 phần tử
```

Chỉ số	0	1	2	3	4
marks					

marks[0] và marks[2] nằm ở vị trí thứ mấy trong mảng?

Mảng 1 chiều (One-dimensional array)

(b) Khởi tạo mảng 1-D

- Khai báo n-1 Array: `data_type identifier[length] = {list of value};`
- Giải thích cú pháp:
 - `data_type` : kiểu dữ liệu của giá trị được lưu trữ trong mảng
 - `identifier`: tên của mảng
 - `length` : số lượng phần tử
 - `list of values`: danh sách các biến được khởi tạo
- Ví dụ

```
int marks[5] = {70, 50, 20, 96, 49};
```

Chỉ số	0	1	2	3	4
marks	70	50	20	96	49

Mảng marks tại chỉ số 3 và 0 mang giá trị là bao nhiêu?

Truy cập đến một phần tử của mảng

(C1) Sử dụng index (tham chiếu)

- Cú pháp:
- Ví dụ

```
arrayName[index];
```

Chỉ số	0	1	2	3	4
marks	70	50	20	96	49

```
int marks[5] = {70,50,20,96,49};
```

```
int sum = marks[2] + marks[0];
```

Giá trị của **sum** bằng bao nhiêu?

Truy cập đến một phần tử của mảng

(C2) Sử dụng con trỏ

- Cú pháp:

```
*(arrayName+i);
```

- Ví dụ

Chỉ số	0	1	2	3	4
marks	70	50	20	96	49

```
int marks[5] = {70,50,20,96,49};
```

```
int sum = *marks + *(marks+3);
```

Giá trị của **sum** bằng bao nhiêu?

Truy cập vào phần tử của mảng

(C3) Sử dụng vòng lặp

- Ví dụ:

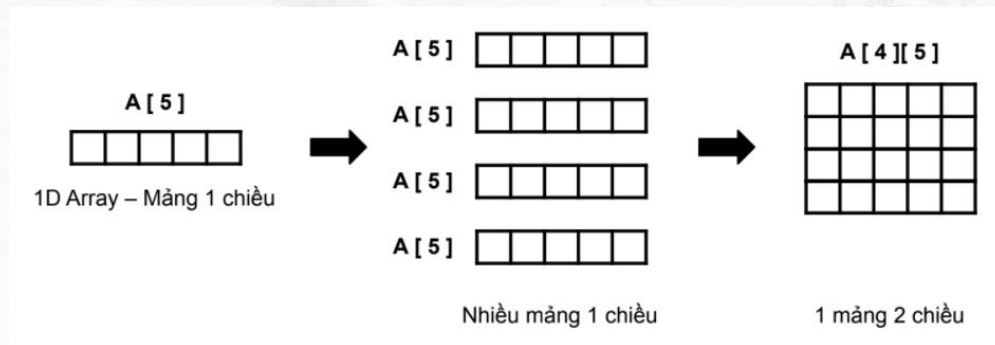
```
int marks[5];  
for (int i = 0; i<=4; i++)  
marks[i] = i;
```

- Kết quả:

Chỉ số	0	1	2	3	4
marks	0	1	2	3	4

MẢNG HAI CHIỀU LÀ GÌ?

- Là ma trận ($m \times n$) của các phần tử có cùng kiểu dữ liệu.
- Mảng 2 chiều `array[m][n]` cấu tạo bởi mảng 1 chiều m phần tử mà mỗi phần tử trong đó là một mảng một chiều kích thước n . Khi đó kích thước của mảng `array` là $m \times n$ (số hàng $\times$ số cột)



KHAI BÁO MẢNG HAI CHIỀU

<kiểu dữ liệu> <tên> [<dòng>][<cột>]

```
int array[5][5];
```


```
double array[3][4] =  
{1,2,3,4,5,6,7,8,9,10,11,12};
```

1	2	3	4
5	6	7	8
9	10	11	12

```
float array[6][8] = {};
```

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

NHẬP GIÁ TRỊ VÀO MẢNG HAI CHIỀU

DÙNG 2 VÒNG LẶP FOR

```
for (int i = 0; i < m; i++)  
    for (int j = 0, j < n, j++)  
    {  
        printf("array[%d][%d] = ", i, j);  
        scanf("%d", &a[i][j]);  
    }
```

cột j		0	1	2	3
hàng i	0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
	1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
	2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Mảng n chiều (n-dimensional array)

- Là ma trận có n chiều, với mỗi phần tử của chiều thứ k chứa một mảng n-k chiều.
- Ví dụ:
 - Mảng 3 chiều: `arr[5][6][7]` có dạng ma trận kích thước 5x6x7
 - Mảng 4 chiều: `arr[2][2][3][4]` có dạng ma trận kích thước 2x2x3x4.
 - ...
- Ma trận n chiều có khả năng chứa lượng cực lớn thông tin mà chúng có mối liên kết chặt chẽ với nhau.

XUẤT MẢNG HAI CHIỀU

DÙNG 2 VÒNG LẶP FOR

```
for (int i = 0; i < m; i++)  
    for (int j = 0, j < n, j++)  
        printf("array[%d][%d] = ", i, j, a[i][j]);
```

CÁCH TRUY CẬP VÀO MẢNG 2 CHIỀU GIỐNG VỚI MẢNG 1 CHIỀU

SEARCH

```
graph TD; SEARCH --> LINEAR_SEARCH; SEARCH --> BINARY_SEARCH;
```

LINEAR SEARCH

CTDL00

BINARY SEARCH

CTDL01

LINEAR SEARCH

Thuật toán tìm kiếm tuyến tính sẽ lần lượt so sánh phần tử cần tìm (key) với từng phần tử trong một cấu trúc dữ liệu theo thứ tự của chỉ số từ nhỏ đến lớn (hoặc theo một chiều nhất định) để tìm ra vị trí của key trong cấu trúc dữ liệu đó.

Ví dụ 1: Tìm số 8 trong dãy số sau:

1 3 -8 8 2 1 2 3

LINEAR SEARCH

Bước 1: Chọn cấu trúc dữ liệu phù hợp với đầu vào

Ở đây, ta có thể dễ dàng nghĩ ngay đến mảng số nguyên. Ta sẽ đưa cả dãy số thành một mảng có độ dài là 8.

0	1	2	3	4	5	6	7
1	3	-8	8	2	1	2	3

Lúc này, ta lưu số cần tìm vào biến kiểu nguyên **key**.

8

LINEAR SEARCH

Bước 2: So sánh key với từng phần tử trong mảng

Tạo biến chạy $i = 0$, ta sẽ so sánh $a[i]$ với key mỗi khi i được làm mới.

$i = 0$, so sánh key với $a[0]$

0	1	2	3	4	5	6	7
1	3	-8	8	2	1	2	3

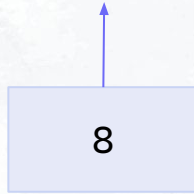
8

Thấy rằng $a[0] = 1$ khác với key nên ta tăng i lên 1, tiếp tục so sánh với phần tử thứ hai.

LINEAR SEARCH

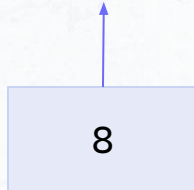
i = 1

0	1	2	3	4	5	6	7
1	3	-8	8	2	1	2	3



i = 2

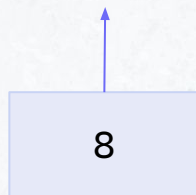
0	1	2	3	4	5	6	7
1	3	-8	8	2	1	2	3



LINEAR SEARCH

i = 3

0	1	2	3	4	5	6	7
1	3	-8	8	2	1	2	3



Khi **key == a[i]**, ta sẽ có được giá trị **i** chính là vị trí của **key** trong mảng.

Ở ví dụ này, số 8 được tìm thấy nằm ở chỉ số 3 trong mảng.

INSERT

Ví dụ 2.1: Chèn số 8 vào mảng sau ở vị trí **cuối mảng**:

1 3 -8 0 2 1 2 3

Ý tưởng:

- Mở rộng mảng từ 8 lên 9 phần tử.
- Gán vào phần tử chỉ số 8.

INSERT

Ví dụ 2.2: Chèn số 8 vào mảng sau ở vị trí **giữa mảng**:

1 3 -8 0 2 1 2 3

Ở ví dụ này, ta giả sử cần chèn số 8 vào **chỉ số 4** của mảng.

Ý tưởng:

- Mở rộng mảng từ 8 lên 9 phần tử.
- Lần lượt di chuyển các phần tử từ chỉ số 4 đến chỉ số 7 tiến lên một ô.
- Chèn số 8 vào ô có chỉ số 4.

INSERT

- Lưu dãy số trên vào một mảng kiểu nguyên a

0	1	2	3	4	5	6	7
1	3	-8	0	2	1	2	3

- Ở cuối mảng (vị trí chỉ số 7), ta mở rộng thêm 1 ô là a[8]

0	1	2	3	4	5	6	7	8
1	3	-8	0	2	1	2	3	

- Gán số 8 vào a[8], ta được mảng a đã chèn.

0	1	2	3	4	5	6	7	8
1	3	-8	0	2	1	2	3	8

INSERT

- Ta di dời phần tử tại chỉ số 6 sang chỉ số 7:

0	1	2	3	4	5	6	7	8
1	3	-8	0	2	1		2	3



- Ta di dời phần tử tại chỉ số 5 sang chỉ số 6:

0	1	2	3	4	5	6	7	8
1	3	-8	0	2		1	2	3



- Ta di dời phần tử tại chỉ số 4 sang chỉ số 5

0	1	2	3	4	5	6	7	8
1	3	-8	0		2	1	2	3



INSERT

- Lưu dãy số trên vào một mảng kiểu nguyên a

0	1	2	3	4	5	6	7
1	3	-8	0	2	1	2	3

- Ở cuối mảng (vị trí chỉ số 7), ta mở rộng thêm 1 ô là a[8]

0	1	2	3	4	5	6	7	8
1	3	-8	0	2	1	2	3	

- Ta di dời phần tử tại chỉ số 7 sang chỉ số 8:

0	1	2	3	4	5	6	7	8
1	3	-8	0	2	1	2		3



INSERT

- Chèn số 8 vào $a[4]$

0	1	2	3	4	5	6	7	8
1	3	-8	0	8	2	1	2	3

Vậy ta vừa chèn số 8 vào mảng tại chỉ số 4.

Chú ý: Trong thuật toán trên, **để dễ hiểu**, ta đã lược bỏ đi chi tiết gán $a[i+1] = a[i]$ thì giá trị tại $a[i]$ không bị mất đi mà vẫn còn giữ nguyên giá trị. Mảng chỉ thật sự “sạch” tại bước cuối cùng, khi $a[i] = \text{key}$ thì các giá trị dư do các phép gán mới thực sự bị triệt tiêu hết. Do đó, hành động “di dời” chỉ tượng trưng cho phép gán chồng các giá trị chứ không thực sự “di dời” các giá trị khỏi hẳn nơi chúng từng ở!

Câu hỏi: Vậy nếu ta cần chèn số 8 vào đầu mảng, ta sẽ thực hiện như thế nào?

DELETE

Ví dụ 2.1: Xoá phần tử tại chỉ số 4 của mảng:

1 3 -8 0 2 1 2 3

Ý tưởng:

- Gán chồng lên các phần tử từ chỉ số 5 xuống 4, từ 6 xuống 5, 7 xuống 6.
- Giảm kích thước của mảng từ 8 xuống 7.

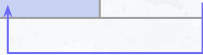
DELETE

- Lưu dãy số trên vào một mảng kiểu nguyên a

0	1	2	3	4	5	6	7
1	3	-8	0	2	1	2	3

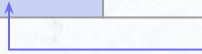
- Dời phần tử tại chỉ số 5 xuống chỉ số 4:

0	1	2	3	4	5	6	7
1	3	-8	0	1		2	3



- Dời phần tử tại chỉ số 6 xuống chỉ số 5:

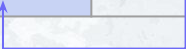
0	1	2	3	4	5	6	7
1	3	-8	0	1	2		3



DELETE

- Dời phần tử tại chỉ số 7 xuống chỉ số 6:

0	1	2	3	4	5	6	7
1	3	-8	0	1	2	3	



- Giảm kích thước mảng từ 8 xuống 7

0	1	2	3	4	5	6
1	3	-8	0	1	2	3

Vậy ta đã được mảng xoá phần tử tại chỉ số 4.

Chú ý: Giống như phần insert ở trên, việc “di dời” cũng không thực sự đúng với minh hoạ của hình ảnh. Trong C, ta dùng phép gán đè lên các phần tử trước đó, rồi sau đó giảm biến kích thước mảng đi 1.

Câu hỏi: Nếu muốn xoá phần tử ở đầu hoặc cuối mảng, ta thực hiện thế nào?

03 →

BÀI TẬP TẠI LỚP

SEARCH - INSERT - DELETE

(AI)

BÀI TẬP TẠI LỚP 1

SEARCH

Hoàn thành hàm `searchArray` với chức năng tìm kiếm một phần tử số có xuất hiện trong mảng hay không, nếu có thì xuất ra chỉ số (index) đầu tiên tìm được.

INPUT: Mảng số nguyên `arr` có kích thước `n` và số nguyên `value`.

OUTPUT: Tìm được tại chỉ số tương ứng hoặc không tìm được

```
#include <stdio.h>

int searchArray(int arr[], int n, int value) {
    int index = -1;

    return index;
}

int main() {
    int n;
    int arr[];
    int value;

    //Input arr and value

    int index = searchArray(arr, n, value);
    if (index == -1)
        printf("Not found!");
    else
        printf("Found %d at %d", value, index);
}
```

TEST CASE BÀI TẬP TẠI LỚP 1

arr	0	1	2
	2	-3	7

TEST	INPUT	OUTPUT
1	3 2 -3 7 7	Found 7 at 2
2	6 1 1 1 1 2 1 1	Found 1 at 0
3	9 1 2 2 4 2 1 4 -4 9 -4	Found -4 at 7
4	9 0 1 1 0 1 1 0 1 0 2	Not found!

BÀI TẬP TẠI LỚP 2

INSERT

Hoàn thành hàm **insertArray** với chức năng chèn một phần tử cho trước tại chỉ số của một mảng số nguyên.

INPUT:

- kích thước mảng **n**
- mảng số nguyên **a**
- số nguyên **value**
- chỉ số **index** cần chèn

OUTPUT: Mảng **a** sau khi chèn.

```
void insertArray(int a[], int *n, int value, int
index);

int main() {
    int n = 5;
    int array[10] = { 3, 1, 5, 7, 4 };
    int value;
    int index = 2;
    value = 8;
    insertArray(a, &n, value, index);    // 3 1 8 5 7 4
}
```

TEST CASE BÀI TẬP TẠI LỚP 2

TEST	INPUT	OUTPUT
1	5 3 1 5 7 4 8 2	3 1 8 5 7 4
2	7 1 2 3 4 5 6 -7 -8 3	1 2 3 -8 4 5 6 -7
3 cuối	9 1 2 2 4 2 1 4 -4 9 10 9	1 2 2 4 2 1 4 -4 9 10
4 đầu	4 -2 5 12 -3 0 0	0 -2 5 12 -3

BÀI TẬP TẠI LỚP 3

DELETE

Hoàn thành hàm **deleteArray** với chức năng xoá một phần tử cho trước tại chỉ số của một mảng số nguyên.

INPUT:

- kích thước mảng **n**
- mảng số nguyên **a**
- chỉ số **index** cần xoá

OUTPUT: Mảng **a** sau khi xoá.

```
#include<stdio.h>
void deleteArray(int a[], int* n, int index){
}
int main(){
    int a[] = {1,4,3,7,5};
    int n = 5;
    int index = 2;

    deleteArray(a,&n,index);
    for(int i = 0; i < n; i++){
        printf("%d ",a[i]);
    }
    printf("\n");
}
```

TEST CASE BÀI TẬP TẠI LỚP 3

DELETE

TEST	INPUT	OUTPUT
1	5 1 4 3 7 5 2	1 4 7 5
2	7 1 2 3 4 5 6 -7 4	1 2 3 4 6 -7
3 cuối	9 1 2 2 4 2 1 4 -4 9 8	1 2 2 4 2 1 4 -4
4 đầu	4 -2 5 12 -3 0	5 12 -3

03 →

BÀI TẬP VỀ NHÀ

SEARCH - INSERT - DELETE

INT - REAL ARRAY

STRING

(AI)

BÀI TẬP VỀ NHÀ

SEARCH

Bài tập 1: Cho một dãy số thực có n phần tử và một số nguyên m . Tìm số có giá trị gần nhất với số m đã cho trong dãy.

INPUT:

- Dòng đầu tiên là số lượng phần tử của dãy n .
- n dòng tiếp theo, mỗi dòng chứa một số thực là phần tử của dãy.
- Dòng cuối cùng là số nguyên m .

OUTPUT: Số tìm được, vị trí (chỉ số) của số đó trong dãy.

TEST	INPUT	OUTPUT
1	6 -1.02 0 1.24 1.32 -2.12 3.23 1	1.24 at index 2 nearest to 1

BÀI TẬP VỀ NHÀ

SEARCH

Bài tập 2: Cho một chuỗi có độ dài chưa biết và một ký tự. Tìm trong chuỗi có xuất hiện ký tự đó không? Nếu có thì xuất ra chỉ số đầu tiên tìm được. (**không** phân biệt ký tự hoa và ký tự thường)

INPUT: Dòng đầu tiên là chuỗi ký tự, dòng thứ hai là một ký tự.

OUTPUT: Tìm được tại chỉ số tương ứng hoặc không tìm được

TEST	INPUT	OUTPUT
1	DataSetAndAlgorithm A	Found A at 1
2	MmMmMmmM m	Found m at 0
3	PhamNhungLam T	Not found!

BÀI TẬP VỀ NHÀ

SEARCH

Bài tập 3: Cho một mảng có kích thước n , mỗi phần tử của mảng là một chuỗi có độ dài đúng bằng k .

Ví dụ: Mảng `serialKey` có độ dài là 4 và độ dài các phần tử chuỗi đúng bằng 8 có dạng:

0	1	2	3
01sd203a	981am780	n1h2u3ng	hvthaott

Yêu cầu: Lưu các chuỗi nhập từ bàn phím vào mảng `serialKey` và viết hàm `search` cho phép tìm một chuỗi ký tự nhập từ bên ngoài có ở trong mảng hay không, nếu có thì ở chỉ số nào. Cài đặt chương trình C để kiểm tra tính hiệu quả của hàm `search`.

INPUT: Dòng đầu tiên là kích thước mảng n , dòng thứ hai là kích thước phần tử k , n dòng sau đó mỗi dòng là một chuỗi có độ dài k , dòng cuối cùng là **chuỗi cần tìm**.

OUTPUT: Tìm được tại chỉ số tương ứng hoặc không tìm được.

TEST CASE BÀI TẬP VỀ NHÀ 3

SEARCH

TEST	INPUT	OUTPUT
1	4 8 01sd203a 981am780 N1h2u3ng hvthaott hvthaott	Found hvthaott at 3
2	5 3 2k3 2k4 2k2 3j3 2k2 2k2	Found 2k2 at 2

BÀI TẬP VỀ NHÀ

INSERT

Bài tập 4: Chèn một ký tự cho trước vào một chuỗi ký tự.

INPUT:

- Dòng đầu tiên là chuỗi ký tự có độ dài không quá 1000
- Dòng thứ hai là ký tự cần chèn và chỉ số tương ứng

OUTPUT: Chuỗi ký tự sau khi chèn.

TEST	INPUT	OUTPUT
1	DataStructureAndAlgorithm A 3	DataAaStructureAndAlgorithm
2	MmMmMmmM m 0	mMmMmMmmM
3	thCTDL&G T 8	thCTDL>

Chú ý: Không sử dụng bất kỳ hàm có sẵn trong thư viện **string.h** ngoại trừ hàm `strlen`. 53

BÀI TẬP VỀ NHÀ

INSERT

Bài tập 5: Cho một mảng có kích thước n , mỗi phần tử của mảng là một chuỗi ký tự có độ dài cố định là m . Hãy chèn một chuỗi ký tự vào chỉ số tương ứng trong mảng đó.

INPUT:

- Dòng đầu tiên là số nguyên n biểu thị kích thước của mảng và số nguyên m đại diện cho kích thước của các chuỗi phần tử.
- n dòng tiếp theo, mỗi dòng là 1 chuỗi có kích thước m .
- Dòng thứ $n + 2$ là chỉ số trong mảng cần chèn.
- Dòng cuối là chuỗi cần chèn vào mảng cũng có kích thước m

OUTPUT: Mảng sau khi chèn.

Chú ý: Mã nguồn nên phân thành các hàm để trình bày rõ ràng thuật toán.

TEST CASE BÀI TẬP VỀ NHÀ 5

INSERT

TEST	INPUT	OUTPUT
1	3 4 abcd mnop efgh 1 ijkl	abcd ijkl mnop efgh
2	9 00applepie 01vnuhcmus 02ppnhungt 03phlamtth 04hvtthaoms 05shjkadkj 06thctdlgt 0716TTHusm 0922KDLusm 8 0822TTHusm	00applepie 01vnuhcmus 02ppnhungt 03phlamtth 04hvtthaoms 05shjkadkj 06thctdlgt 0716TTHusm 0822TTHusm 0922KDLusm

BÀI TẬP VỀ NHÀ

INSERT

Bài tập 6: Cho một mảng số thực có n phần tử và một danh sách các số thực khác cần chèn vào mảng đó. Hãy chèn **cùng lúc** tất cả các số từ danh sách vào mảng số thực với các chỉ số tương ứng.

INPUT:

- Dòng đầu tiên là số nguyên n biểu thị kích thước của mảng a .
- Dòng thứ hai là a .
- Dòng thứ ba là số nguyên m biểu thị số lượng số cần chèn vào mảng a .
- m dòng tiếp theo, mỗi dòng chứa 2 số, số đầu tiên là số thực cần chèn, số thứ hai là chỉ số cần chèn vào mảng a .

OUTPUT: Mảng a sau khi chèn.

Chú ý: Mã nguồn nên phân thành các hàm để trình bày rõ ràng thuật toán.

TEST CASE BÀI TẬP VỀ NHÀ 6

INSERT

TEST	INPUT	OUTPUT
1	5 2.2 -3.0 19.12 -10.22 9.0 3 3.0 1 -2.5 3 0.0 4	2.2 3.0 -3.0 19.12 -2.5 -10.22 0.0 9.0
2	10 0.2 0.3 0.5 0.7 0.8 1.0 1.2 1.3 1.4 1.6 7 0.1 0 0.4 2 0.6 3 0.9 5 1.1 6 1.5 9 1.7 10	0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0 1.1 1.2 1.3 1.4 1.5 1.6 1.7

BÀI TẬP VỀ NHÀ

INSERT

Bài tập 7: (**APPLE PIE**) Bộ phận phát triển sản phẩm của một công ty chuyên sản xuất bánh táo vừa nhận được một danh sách các mã sản phẩm của từng chiếc bánh táo. Mỗi mã sản phẩm là một chuỗi có độ dài cố định là 10 ký tự chỉ bao gồm chữ hoặc số, với hai ký tự đầu là chỉ số thứ tự cần lưu trong danh sách. Danh sách được cung cấp là một **mảng** mà mỗi phần tử là một chuỗi ký tự mã sản phẩm, với hai ký tự đầu có thể không tương ứng với chỉ số hiện tại của mảng.

YÊU CẦU: Lưu danh sách vào mảng và bổ sung các mã sản phẩm còn thiếu vào mảng sao cho hai ký tự đầu của tất cả mã sản phẩm đều trùng khớp với chỉ số của mảng.

INPUT:

- Dòng đầu tiên là số nguyên dương n đại diện cho kích thước của mảng hiện tại.
- n dòng tiếp theo, mỗi dòng là một chuỗi ký tự đại diện cho phần tử của mảng.
- Dòng thứ $n + 2$ là số nguyên dương m là số lượng chuỗi cần chèn vào mảng.
- m dòng tiếp theo, mỗi dòng là một chuỗi ký tự

OUTPUT: Mảng sau khi chèn.

TEST CASE BÀI TẬP VỀ NHÀ 7

INSERT

TEST	INPUT	OUTPUT
1	2 01dkshgbkb 02kshdkjkhk 2 03jkashjhj 00djhkfdhf	00djhkfdhf 01dkshgbkb 02kshdkjkhk 03jkashjhj
2	6 01vnuhcmus 03phlamtth 04hvtthaoms 05shjkadkj 0716TTHusm 0922KDLusm 4 02ppnhungt 00applepie 0822TTHusm 06thctdlgt	00applepie 01vnuhcmus 02ppnhungt 03phlamtth 04hvtthaoms 05shjkadkj 06thctdlgt 0716TTHusm 0822TTHusm 0922KDLusm

BÀI TẬP VỀ NHÀ

DELETE

Bài tập 8: Xóa một ký tự cho trước chỉ số của một chuỗi ký tự.

INPUT:

- Dòng đầu tiên là chuỗi ký tự có độ dài không quá 1000
- Dòng thứ hai là chỉ số của phần tử cần xóa.

OUTPUT: Chuỗi ký tự sau khi xóa.

TEST	INPUT	OUTPUT
1	DataStructureAndAlgorithm 3	DatStructureAndAlgorithm
2	MmMmMmmM 7	MmMmMmm
3	thCTDL> 8	thCTDL&G

Chú ý: Không sử dụng bất kỳ hàm có sẵn trong thư viện **string.h** ngoại trừ hàm `strlen`. 60

BÀI TẬP VỀ NHÀ

SEARCH

DELETE

Bài tập 9: Cho một mảng mà mỗi phần tử của mảng là một cặp số thực đại diện cho tọa độ các mỏ đá quý tìm được trên một khu vực. Người dùng lưu trữ các tọa độ này để lập trình cho robot khai thác sau này. Mỗi khi khai thác xong một mỏ đá quý, máy tính sẽ xóa tọa độ tương ứng trong mảng.

Yêu cầu: Lưu trữ các cặp số thực vào mảng và xóa đi một cặp số thực được người dùng cung cấp.

Input:

- Dòng đầu tiên chứa số tự nhiên n là kích thước của mảng
- n dòng tiếp theo, mỗi dòng là một cặp số thực là các phần tử của mảng cách nhau bởi một khoảng trống.
- Dòng cuối cùng chứa một cặp số thực được yêu cầu xóa khỏi mảng.

Output: Mảng sau khi xóa kèm theo chỉ số đã xóa.

TEST CASE BÀI TẬP VỀ NHÀ 9

SEARCH

DELETE

TEST	INPUT	OUTPUT
1	3 1.4 2.74 -2.3 9.2 0.5 -0.5 -2.3 9.2	1.4 2.74 0.5 -0.5 Delete -2.3 9.2 at 1
2	5 -0.87878 1.274 76.498 -10.938793 8237.20 3873.2 76.34 9.676 8237.20 3873.2 8237.20 3873.2	-0.87878 1.274 76.498 -10.938793 76.34 9.676 Delete 8237.20 3873.2 at 2, 4
3	4 1 1 2 2 3 3 4 4 5 5	1 1 2 2 3 3 4 4 Can not delete 5 5 from the array!

BÀI TẬP TẠI VỀ NHÀ

SEARCH

Bài tập 10.1: (POISON ATTACK - level 1)

Một nhà nghiên cứu về bảo mật thực hiện một thí nghiệm mô phỏng về các vụ tấn công vào hệ thống bằng mã độc. Các máy chủ được nhận những gói thông tin dạng chuỗi ký tự (char) và được yêu cầu kiểm tra các chuỗi đó có mã độc hay không. Mã độc là một chuỗi ký tự con có độ dài nhỏ hơn so với độ dài của gói tin, được cung cấp trong danh sách tùy vào cấp độ của thí nghiệm.

Yêu cầu: Viết chương trình (phân hàm con) cho phép người dùng nhập vào một gói tin là một chuỗi có độ dài tối đa 1000 ký tự và một danh sách các chuỗi con (không quá 5) được xem là mã độc. **Chương trình gửi thông báo rằng gói tin có mã độc hay không.**

INPUT:

- Dòng đầu tiên là chuỗi ký tự có độ dài không quá 1000 ký tự.
- Dòng thứ hai là số nguyên dương $0 < k < 6$ là số lượng mã độc trong danh sách.
- k dòng tiếp theo là các chuỗi trong danh sách mã độc.

OUTPUT: Số lượng và vị trí của mã độc trong dãy.

TEST CASE 10.1

SEARCH

TEST	INPUT	OUTPUT
1	gdfjhhghjhgdsfjhhag 1 hhg	1 hhg 4
2	hdsa2023u872hdsa2023hs726ctdlctdlgfd7w7sgd8 9a7dsa76s82ctdl2023j92jcdt2 2 ctdl dsa2023	3 ctdl 14, 18, 43 2 dsa2023 1, 13
3	89324y7w9ui3jbwediugd78uawihdioyqwt8eh1298e y89128uyeiuqwhbdoijwq89dhawuidhn98qwdj98 32897jkwhdhsa 3 hvthao ppnhung phlam	0 hvthao 0 ppnhung 0 phlam

Chú ý: Không được dùng bất kỳ hàm có sẵn nào trong thư viện <string.h> trừ strlen

BÀI TẬP TẠI VỀ NHÀ

DELETE

Bài tập 10.2: (POISON ATTACK - level 2)

Các nghiên cứu sinh được yêu cầu cải tiến mã nguồn từ level 1 bằng cách xoá đi tất cả mã độc có trong gói tin. Đó được gọi là quá trình làm sạch dữ liệu.

Yêu cầu: Cải tiến chương trình ở **Bài tập 10.1**, thêm hàm với chức năng xoá đi tất cả mã độc tìm được trong chuỗi nhập vào và xuất ra chuỗi vừa được làm sạch.

INPUT:

- Dòng đầu tiên là chuỗi ký tự có độ dài không quá 1000 ký tự.
- Dòng thứ hai là số nguyên dương $0 < k < 6$ là số lượng mã độc trong danh sách.
- k dòng tiếp theo là các chuỗi trong danh sách mã độc.

OUTPUT: Chuỗi đã được làm sạch.

TEST CASE 10.2

DELETE

TEST	INPUT	OUTPUT
1	gdfjhhghjhgdsfjhhag 1 hhg	gdfjhjhgdsfjhhag
2	hdsa2023u872hdsa2023hs726ctdlctdlgfd7w7sgd8 9a7dsa76s82ctdl12023j92jcdt2 2 ctdl dsa2023	hu872hhs726gfd7w7sgd89a7dsa76s822023j92jcdt2
3	89324y7w9ui3jbwediugd78uawihdionyqwt8eh1298e y89128uyeigqwhbdoijwq89dhawuidhn98qwdj98 32897jkwhdhsa 3 hvthao ppnhung phlam	89324y7w9ui3jbwediugd78uawihdionyqwt8eh1298ey89 128uyeigqwhbdoijwq89dhawuidhn98qwdj98 32897jkwhdhsa

Chú ý: Không được dùng bất kỳ hàm có sẵn nào trong thư viện <string.h> trừ strlen

BÀI TẬP TẠI VỀ NHÀ

INSERT

Bài tập 10.3: (POISON ATTACK - level 3)

Quá trình làm sạch dữ liệu vô tình làm rút ngắn chiều dài gói tin, khiến cho các giao thức trong bộ nhớ bị nhiễu loạn nên nhà nghiên cứu yêu cầu các nghiên cứu sinh không xóa đi tất cả mã độc trong gói tin nữa, thay vào đó sẽ chèn sau mỗi ký tự trong mã độc là một ký tự mới liền kề với ký tự đó trong bảng mã Ascii. Việc này sẽ làm biến dạng hoá mã độc, khiến chúng mất đi chức năng của chúng.

Yêu cầu: Cải tiến chương trình ở **Bài tập 10.1**, thêm hàm với chức năng chèn sau mỗi ký tự trong mã độc là một ký tự liền kề theo bảng mã ascii.

INPUT:

- Dòng đầu tiên là chuỗi ký tự có độ dài không quá 1000 ký tự.
- Dòng thứ hai là số nguyên dương $0 < k < 6$ là số lượng mã độc trong danh sách.
- k dòng tiếp theo là các chuỗi trong danh sách mã độc.

OUTPUT: Chuỗi đã được làm sạch.

TEST CASE 10.3

INSERT

TEST	INPUT	OUTPUT
1	gdfjhhghjhgdgsfjhhag 1 hhg	gdfjhihighjhgdgsfjhhag
2	hdsa2023u872hdsa2023hs726ctdlctdlgfd7w7sgd8 9a7dsa76s82ctdl2023j92jcdt2 2 ctdl dsa2023	hdestab23012334u872hdestab23012334hs726cdtudel mcdtudelmgfd7w7sgd89a7dsa76s82cdtudelm2023j92j cdt2
3	89324y7w9ui3jbwediugd78uawihdionyqwt8eh1298e y89128uyeigqwhbdoijwq89dhawuidhn98qwdj98 32897jkwhdhsa 3 hvthao ppnhung phlam	89324y7w9ui3jbwediugd78uawihdionyqwt8eh1298ey89 128uyeigqwhbdoijwq89dhawuidhn98qwdj98 32897jkwhdhsa

Chú ý: Không được dùng bất kỳ hàm có sẵn nào trong thư viện <string.h> trừ strlen

THE END



If you have any question, please
contact us at email:

ta.tinth.hcmus@gmail.com

